

# Pre-Search Checklist — Clone Synthesis Tutor

Complete this before writing code. Save your AI conversation as a reference document.

*Project: Clone Synthesis Tutor · Prepared by: val · Date: 2026-05-18*

## Phase 1: Define Your Constraints

### 1. Domain Selection

- **Which domain: healthcare, insurance, finance, legal, or custom?**

Custom — ed-tech / elementary-math tutoring. Target concept is fraction equivalence (e.g.,  $1/2 = 2/4$ ), per the project brief.

- **What specific use cases will you support?**

One scrollytelling-style lesson: a full-page narrative (Jupyter-notebook feel) where authored prose flows top-to-bottom and embedded interactive checkpoints anchor the scroll.

Recommended shape for the 1-week build: ~5 story beats, 3 fraction-block manipulatives, 3 multiple-choice bubble checks, and 1–2 optional open-ended 'what do you notice?' reflection moments. The student completes each checkpoint before the next prose chunk reveals.

- **What are the verification requirements for this domain?**

Math correctness is deterministic and gated server-side: MC bubble answers are checked against a preselected key; manipulative end-states are compared to expected fraction values. Open-ended reflection beats are non-blocking (Haiku reacts in-character, but doesn't grade). Pedagogical tone (warm, encouraging, age-appropriate) is verified by sample-review of session transcripts during the build week.

- **What data sources will you need access to?**

No external data sources. Inputs are the authored lesson script (JSON: beats + their text + their checkpoint widgets + their MC keys), the live manipulative state, and the student's bubble selections / reflection text. No student PII storage in v1.

### 2. Scale & Performance

- **Expected query volume?**

Prototype-scale: 1 student per session, demo runs only during build week + final demo. Inferred: <50 total lesson runs across development and the demo video.

- **Acceptable latency for responses?**

Inferred budgets: narrator TTS first-audio-out p95 < 800ms after a beat reveals (pre-generate paragraph audio one beat ahead to mask this entirely); manipulative animations 60fps (<16ms frame budget); LangGraph node transitions < 500ms; Haiku hint generation after a wrong MC p95 < 1.2s (show a 'thinking...' state if exceeded). The whole feel must read as 'instant scroll-and-respond.'

- **Concurrent user requirements?**

Single user (one student on one iPad). No concurrency requirements; the provided iPad is the only target device for the demo.

- **Cost constraints for LLM calls?**

Not a constraint at prototype scale. Inferred per-lesson cost: ElevenLabs TTS-only (no Conversational AI / no STT) ~\$0.18/1k characters × ~5,000 chars per lesson ≈ \$0.90 voice; Haiku 4.5 across ~10k tokens of paraphrase + hints + reflection reactions at \$1/\$5 per Mtok ≈ <\$0.05. Roughly \$1 per full lesson run.

### 3. Reliability Requirements

- **What's the cost of a wrong answer in your domain?**

Demo-level: low absolute cost (no real student outcomes at stake), but high reputational cost — a tutor that affirms an incorrect MC bubble or whose narrator sounds robotic kills the 'feels like exploration, not homework' goal from the brief. Treat tutor errors as demo-blocking bugs.

- **What verification is non-negotiable?**

Deterministic check of MC bubble selections and manipulative end-states against the authored key, before any 'great job!' response or any next-beat reveal. Haiku's paraphrased praise / hint never overrides the deterministic verdict.

- **Human-in-the-loop requirements?**

Not in the runtime loop — the lesson must run unattended on the iPad. In the build loop: Patrick (technical contact) reviews session transcripts mid-week to flag pedagogical missteps before final demo.

- **Audit/compliance needs?**

Light. Inferred: store local session transcripts (beat order, MC selections, manipulative state diffs, Haiku calls, reflection texts) for the build week to support iteration and the 1–2 min demo video. No formal compliance regime; recommend confirming with Patrick before any actual-child testing.

### 4. Team & Skill Constraints

- **Familiarity with agent frameworks?**

*Not specified — recommend confirming. Inferred from stack choice (LangGraph): solo builder is comfortable enough with LangGraph state-machine patterns to author the lesson as a node graph rather than a free-form chain.*

- **Experience with your chosen domain?**

*Not specified — recommend confirming. Ed-tech / fractions pedagogy is well-documented (Synthesis Tutor public materials, fraction-bar canonical curricula); plan to lean on the Brainlift research doc cited in the brief.*

- **Comfort with eval/testing frameworks?**

*Not specified — recommend confirming. Inferred: comfortable with pytest / vitest; LLM-eval tooling (LangSmith datasets) will likely be net-new and worth a half-day of upfront investment.*

## Phase 2: Architecture Discovery

### 5. Agent Framework Selection

- **LangChain vs LangGraph vs CrewAI vs custom?**

LangGraph (user decision). Fit is excellent: scrollytelling beats map 1:1 to LangGraph nodes; edges encode 'next beat depends on MC result / manipulative state / reflection sentiment.' The brief's 'simple branching logic' requirement is exactly LangGraph's sweet spot.

- **Single agent or multi-agent architecture?**

Single agent. One LangGraph instance owns the lesson; the manipulative and the MC bubble component are UI surfaces, not sibling agents. Multi-agent adds coordination cost with no payoff for a 1-week scripted lesson.

- **State management requirements?**

LangGraph state holds: `current_beat_id`, `beat_history`, `lesson_phase` (intro | explore | guided | check), per-beat completion status, MC selections, manipulative snapshots, reflection-text history, hint-attempt counter per checkpoint, adaptive-pacing flag (whether the student is breezing or struggling). Persist to local storage so a page reload resumes mid-lesson cleanly during demo.

- **Tool integration complexity?**

Low. ~6 internal tools (see Section 7). No external APIs beyond ElevenLabs TTS and the Anthropic API. UI ↔ agent communication via WebSocket events carrying state diffs. Inferred frontend stack: React + dnd-kit (excellent iPad touch support, pointer events, accessibility) + Framer Motion (spring physics for combine/split/'smash' animations) + a lightweight scroll-snap library or hand-rolled IntersectionObserver for the notebook scroll behavior. Considered and rejected: react-konva/PixiJS (canvas overhead not justified for ~10 draggable blocks), HTML5 native drag-drop (broken on iOS Safari touch), Yjs (no collaboration requirement).

### 6. LLM Selection

- **GPT-5 vs Claude vs open source?**

Claude Haiku 4.5 (user decision). Four roles in this design: (1) drive the LangGraph as a narrator persona — pick the next authored beat based on student performance (adaptive pacing); (2) generate fresh hint copy when a student picks a wrong MC bubble, referencing what they actually did on the blocks; (3) paraphrase narrator prose lightly across sessions so two runs don't read identically; (4) react in-character to open-ended 'what do you notice?' reflection text. All four operate over an authored beat set, not freeform — this keeps the brief's 'scripted dialogue' contract intact.

- **Function calling support requirements?**

Required and central. Haiku must reliably select among ~6 tool calls (`advance_to_beat`, `generate_hint`, `paraphrase_paragraph`, `classify_reflection`, `validate_mc`, `read_manipulative_state`). No parallel tool use or deep chaining required.

- **Context window needs?**

Small. Per-call context is the current beat, last 2–3 beats of student activity, the relevant manipulative snapshot — well under 10k tokens. Haiku's 200k window is overkill, which is fine.

- **Cost per query acceptable?**

Yes — Haiku 4.5 at ~\$1/\$5 per Mtok puts a full lesson well under \$0.05 in LLM costs. ElevenLabs TTS dominates the per-lesson bill; LLM cost is not a design constraint.

## 7. Tool Design

- **What tools does your agent need?**

validate\_mc(beat\_id, selected\_option) → deterministic correct/incorrect against the authored key. read\_manipulative\_state() → current fraction blocks + their fractional values. validate\_manipulative(beat\_id) → deterministic check the student's end-state matches the expected fraction relation. generate\_hint(beat\_id, attempt\_count, manipulative\_snapshot) → Haiku-generated hint referencing the student's actual blocks. paraphrase\_paragraph(beat\_id) → Haiku rewrites the authored prose with small variations for replay value. classify\_reflection(text) → Haiku categorizes a student's open-ended observation (on-topic insight / partial / off-topic) and generates a warm in-character reaction; never gates progression. advance\_to\_beat(next\_beat\_id) → state transition, optionally chosen by Haiku for adaptive pacing.

- **External API dependencies?**

ElevenLabs TTS (text-to-speech only; no Conversational AI, no STT) and Anthropic API (Haiku 4.5). Both required for the demo. Pre-generate the next beat's audio one beat ahead to hide TTS latency behind the student's interaction with the current checkpoint.

- **Mock vs real data for development?**

Not applicable in the usual sense — there is no production data source. Author the lesson script JSON (beats + their prose + their checkpoint widgets + their MC keys + their hint-seed lines) on day 1; build a labeled set of ~20 reflection-text examples per category (on-topic / partial / off-topic) to drive the classify\_reflection eval.

- **Error handling per tool?**

validate\_mc and validate\_manipulative are pure-function — cannot fail. generate\_hint / paraphrase\_paragraph timeout → fall back to the canonical authored hint / paragraph text; lesson remains playable. classify\_reflection failure → drop the reflection beat to a generic 'great observation!' line and continue. ElevenLabs TTS failure → silently degrade to text-only (the paragraph is on the page anyway); never block the lesson.

## 8. Observability Strategy

- **LangSmith vs Braintrust vs other?**

LangSmith. Native LangGraph integration means zero-config traces of every beat transition and tool call — invaluable for debugging the scroll-and-checkpoint flow during the build week. (In-scope per user decision.)

- **What metrics matter most?**

Beat-completion rate (do students reach the end?), MC accuracy per checkpoint, time-per-beat (where do students stall?), hint-trigger rate (how often does the wrong-answer path fire?), reflection-beat opt-in rate (do students engage with open-ended prompts?), total LLM + TTS cost per lesson.

- **Real-time monitoring needs?**

Not strictly real-time. LangSmith live-trace view during development is enough; no on-call paging for a prototype.

- **Cost tracking requirements?**

Yes — Haiku token counts and ElevenLabs character counts per session, surfaced in a simple LangSmith dashboard. Useful both for the brief's cost question and as a data point for any future production conversation.

## 9. Eval Approach

- **How will you measure correctness?**

Two-tier eval (in-scope per user decision). (1) Automated: a labeled set runs on every prompt/script change — MC validation is trivial (deterministic), `classify_reflection` scored against ~60 labeled reflection examples (3 categories × ~20 examples), `generate_hint` scored on a 1–5 rubric by an LLM-as-judge with Haiku-large or Sonnet. (2) Human: Patrick (or another reviewer) scores 5 full session transcripts on a rubric — warmth, pedagogical accuracy of hints, smoothness of scroll-and-reveal flow, in-character quality of reflection reactions.

- **Ground truth data sources?**

Hand-authored labeled reflection set + hand-written 'good hint vs bad hint' pairs for the LLM-as-judge rubric, both built on build-day 1. Ground truth for math correctness is trivially derivable.

- **Automated vs human evaluation?**

Both. Automated gates per-PR changes to prompts/script; human rubric runs once mid-week and once before the final demo recording.

- **CI integration for eval runs?**

Yes. GitHub Actions runs the LangSmith dataset eval on every PR; merge is blocked if `classify_reflection` accuracy drops below 85% or any high-stakes case (MC incorrect → 'great job!' bug) regresses at all.

## 10. Verification Design

- **What claims must be verified?**

Every 'correct' verdict shown to the student must come from `validate_mc` or `validate_manipulative` — both deterministic. Haiku-paraphrased praise and hints are allowed to vary the wording but never to override the verdict. Reflection-beat reactions never gate progression and never assert a correctness claim about math.

- **Fact-checking data sources?**

The authored lesson script (MC keys + expected manipulative end-states) — the source of truth. No external fact-checking; math is closed-form and answers are preselected.

- **Confidence thresholds?**

`classify_reflection` confidence < 0.6 → drop the reflection beat into a safe 'I love that you're thinking about it — let's keep exploring' canonical line rather than risk an off-tone reaction. MC and manipulative checks have no confidence dimension (deterministic).

- **Escalation triggers?**

Three consecutive wrong MC attempts on the same checkpoint → route to a 'let's work it out together' authored beat with a fully worked example using the manipulative (mirrors the brief's branching-logic requirement). Adaptive-pacing flag set to 'struggling' so subsequent beats favor shorter prose chunks and earlier hint surfacing.

## Phase 3: Post-Stack Refinement

### 11. Failure Mode Analysis

- **What happens when tools fail?**

ElevenLabs TTS outage → narrator silently switches to text-on-page only; the lesson stays fully playable because the prose is already rendered. Anthropic API outage → all Haiku-generated copy (hints, paraphrase, reflection reactions) falls back to the authored canonical text. `validate_mc` and `validate_manipulative` are pure-function and cannot fail.

- **How to handle ambiguous queries?**

Almost eliminated by design — primary inputs are bubble taps and drag-drop, both unambiguous. The only ambiguity surface is open-ended reflection text, which routes through `classify_reflection`'s low-confidence canonical fallback (see Section 10).

- **Rate limiting and fallback strategies?**

Demo-scale traffic is well below ElevenLabs and Anthropic free-tier rate limits. Single retry-with-backoff on transient 5xx; surface a brief 'thinking...' UI state if a hint generation takes longer than 800ms.

- **Graceful degradation approach?**

Every Haiku-generated line has a canonical authored fallback. Every TTS audio file has the on-page paragraph as fallback. The lesson must remain completable end-to-end with both Haiku and ElevenLabs disabled — this is the demo-safety contract.

### 12. Security Considerations

- **Prompt injection prevention?**

Low risk surface (target user is a 7–10yo, primary inputs are bubble taps and drag-drop). The only injection vector is reflection-text free input — treat as data inside delimited tags in the system prompt; never let it control tool selection.

- **Data leakage risks?**

Reflection text may contain a child's first name or context. Inferred: confirm Anthropic ZDR (zero-data-retention) and ElevenLabs privacy settings before any session involving an actual minor. Recommend confirming with Patrick whether the demo iPad will be used with real children or only adult demoers.

- **API key management?**

Local `.env.local` for `ANTHROPIC_API_KEY` and `ELEVENLABS_API_KEY`; not committed. Production-style secret rotation out of scope for a 1-week prototype.

- **Audit logging requirements?**

Local-only session transcripts (beat order, MC selections, manipulative state diffs, Haiku calls, reflection texts) written to disk per session, for build-week debugging and demo-video review. No

external audit pipeline.

### 13. Testing Strategy

- **Unit tests for tools?**

Yes. `validate_mc` and `validate_manipulative` get the most coverage — every authored beat's correct + incorrect cases. `paraphrase_paragraph` and `generate_hint` have snapshot tests (do they call Haiku with the expected prompt structure?), not output-equality tests.

- **Integration tests for agent flows?**

Yes. Golden-path test (student answers everything correctly across all 5 beats), plus four failure paths: (a) two wrong MC attempts then correct, (b) manipulative end-state wrong → hint → correct, (c) Haiku timeout → canonical hint surfaces, (d) ElevenLabs timeout → text-only mode engages cleanly. Runs in CI on every PR.

- **Adversarial testing approach?**

*Out of scope per user decision. Surface area is small (taps + drag-drop + short reflection text from a 7–10yo) and the demo doesn't ship to real children. Flag for any future production hardening.*

- **Regression testing setup?**

LangSmith dataset eval gated on PR merge (see Section 9); the labeled reflection set and the hint-rubric set are replayed against any prompt/script edit.

### 14. Open Source Planning

- **What will you release?**

*Out of scope — internal prototype for the 1-week challenge (SuperBuilders.school). Not planned for open-source release.*

- **Licensing considerations?**

*Out of scope — see above.*

- **Documentation requirements?**

README is a deliverable per the brief — must cover how to run the app and a brief overview of the technical approach (LangGraph beat-node graph, ElevenLabs TTS integration, Haiku's four roles, manipulative architecture, scrollytelling UX pattern).

- **Community engagement plan?**

*Out of scope — see above.*

### 15. Deployment & Operations

- **Hosting approach?**

Web app on Vercel (Next.js or Vite + React frontend); LangGraph backend either co-located as Vercel serverless functions or on Render/Modal if cold starts hurt the hint-generation latency budget. iPad accesses via Safari at the preview URL. ElevenLabs TTS calls happen server-side so the API key stays off the client.

- **CI/CD for agent updates?**

GitHub Actions → run unit tests + LangSmith eval → Vercel preview deploy on PR → manual promote to main demo URL. No canary; no automated rollout staging.

- **Monitoring and alerting?**

LangSmith for trace/cost/quality; Vercel built-in metrics for HTTP errors. No paging — builder watches the LangSmith dashboard during demo runs.

- **Rollback strategy?**

Git revert + Vercel redeploy. Pin a 'last-known-good' tag before the final demo recording so any rollback is one click.

## 16. Iteration Planning

- **How will you collect user feedback?**

Self + Patrick during the build week; the 1–2 minute demo video is the primary feedback artifact. Inferred: if any children test the demo, run a short post-lesson interview ('which part was the most fun / where did you get stuck?') alongside session-transcript review.

- **Eval-driven improvement cycle?**

Add any observed misclassified reflection or weak hint to the labeled set immediately; re-run the LangSmith eval before re-recording the demo video. Tight build-week loop, not a long-term cadence.

- **Feature prioritization approach?**

Brief's three functional requirements (conversational tutor, interactive manipulative, lesson flow with check-for-understanding) are the only priorities, instantiated here as scrollytelling beats + 3 manipulatives + 3 MC checks. Cut anything that doesn't strengthen the 1–2 minute demo.

- **Long-term maintenance plan?**

*Out of scope — 1-week sprint deliverable, not a maintained product. Flag for the next conversation if SuperBuilders wants to extend the prototype to a real curriculum.*